ACTIVIDAD 05

Algoritmo genético con tres codificaciones del cromosoma aplicado a la programación de un taller de maquinado (Job-Shop)

Alumno: Alonzo Lara Flores Fecha: 3 de noviembre de 2026

1. Descripción del problema a ser analizado y tratado con un algoritmo genético

Una planta de manufactura por pedido produce piezas metálicas bajo un esquema de **taller de rutas variables (job-shop)**. En la semana se liberan **15 órdenes de trabajo** (J1 a J15) que deben pasar por **6 centros de trabajo**: Corte, Torno, Fresado, Taladrado, Rectificado e Inspección. Cada orden tiene su propia ruta (todas inician en Corte y terminan en Inspección; las máquinas intermedias y su orden cambian según la pieza), lo que da un total de **86 operaciones** con tiempos de proceso de 5 a 60 minutos.

El problema consiste en decidir en qué orden atiende cada máquina las operaciones que le llegan, de modo que todas las órdenes se terminen lo antes posible. Es un problema de programación de la producción (scheduling) clasificado como NP-difícil (Garey, Johnson y Sethi, 1976): el número de secuencias posibles crece de forma factorial, y las reglas de despacho simples (por ejemplo, tiempo de proceso más corto) dejan mucho tiempo ocioso en las máquinas. Por eso es un candidato natural para un algoritmo genético (AG).

Supuestos de la instancia. Los datos son sintéticos pero realistas; se generaron con una semilla fija (2026) para que el experimento sea reproducible:

Supuesto	Descripción
Tiempos de proceso	Enteros en minutos, aleatorios dentro de un rango por máquina: Corte 10–30, Torno 20–60, Fresado 20–60, Taladrado 15–50, Rectificado 15–55 e Inspección 5–20 (incluyen preparación).
Rutas	Corte primero e Inspección al final; entre ambas, 3 o 4 de las 4 máquinas intermedias en orden aleatorio (cada pieza lleva su propio proceso).
Capacidad	Una sola máquina por centro de trabajo; procesa una operación a la vez y no se interrumpe una vez iniciada.
Precedencia	Cada orden sólo puede estar en una máquina a la vez y sus operaciones siguen el orden de su ruta.
Disponibilidad	Las 15 órdenes y las 6 máquinas están disponibles en el instante 0; sin fallas, mantenimiento ni tiempos de traslado; almacenes intermedios ilimitados.

Tabla 1. Supuestos del problema.

Datos de entrada. La Tabla 2 muestra la ruta de cada orden con su tiempo de proceso entre paréntesis (minutos).

Orden	Ruta: máquina (tiempo en min)
J1	Corte(29) → Taladrado(17) → Fresado(47) → Rectificado(50) → Torno(45) → Inspección(10)
J2	Corte(12) → Fresado(32) → Taladrado(33) → Torno(24) → Rectificado(36) → Inspección(17)
J3	Corte(15) → Fresado(45) → Rectificado(48) → Taladrado(15) → Torno(26) → Inspección(16)
J4	Corte(11) → Fresado(43) → Taladrado(24) → Rectificado(19) → Torno(32) → Inspección(10)
J5	Corte(11) → Taladrado(49) → Torno(45) → Fresado(28) → Rectificado(18) → Inspección(20)
J6	Corte(28) → Fresado(30) → Torno(56) → Taladrado(18) → Inspección(16)
J7	Corte(28) → Taladrado(22) → Rectificado(47) → Torno(27) → Fresado(45) → Inspección(11)

Orden	Ruta: máquina (tiempo en min)
J8	Corte(14) → Fresado(58) → Rectificado(32) → Torno(45) → Taladrado(45) → Inspección(16)
J9	Corte(13) → Torno(58) → Rectificado(27) → Taladrado(18) → Fresado(51) → Inspección(19)
J10	Corte(30) → Fresado(22) → Torno(28) → Taladrado(46) → Inspección(11)
J11	Corte(26) → Taladrado(33) → Rectificado(54) → Torno(35) → Fresado(20) → Inspección(18)
J12	Corte(18) → Torno(38) → Taladrado(26) → Rectificado(41) → Inspección(16)
J13	Corte(19) → Torno(48) → Fresado(44) → Rectificado(51) → Taladrado(46) → Inspección(5)
J14	Corte(14) → Taladrado(34) → Fresado(37) → Torno(40) → Rectificado(55) → Inspección(18)
J15	Corte(15) → Taladrado(31) → Rectificado(34) → Torno(27) → Inspección(15)

Tabla 2. Instancia sintética: 15 órdenes, 6 máquinas, 86 operaciones.

Máquina	Operaciones	Carga total (min)
Torno	15	574
Rectificado	13	512
Fresado	13	502
Taladrado	15	457
Corte	15	283
Inspección	15	218

Tabla 3. Carga de trabajo por máquina. La más cargada (Torno, 574 min) fija una cota inferior para el tiempo total.

2. Objetivo del estudio

Objetivo general. Minimizar el tiempo total de terminación del taller (**makespan**, C_max) con un algoritmo genético y determinar cuál de tres codificaciones del cromosoma (binaria, permutación con repetición y llaves aleatorias) produce mejores calendarios.

Objetivos específicos.

- Implementar el mismo AG con las tres codificaciones, dejando iguales el decodificador, el tamaño de población, la selección, el elitismo y el presupuesto de evaluaciones.
- Medir calidad de la solución (mejor, media, peor, desviación), velocidad de convergencia y costo de cómputo en 30 corridas independientes por codificación.
- Comparar los resultados contra una búsqueda aleatoria con el mismo número de evaluaciones, contra reglas de despacho clásicas y contra la cota inferior.
- Verificar con la prueba U de Mann–Whitney ($\alpha = 0.05$) si las diferencias entre codificaciones son estadísticamente significativas.

Función objetivo. Minimizar C_max = máx { C_j }, donde C_j es el instante en que termina la última operación de la orden j. La aptitud de un individuo es f = 1 / C_max. La **cota inferior** es LB = máx { carga de la máquina más cargada, longitud de la ruta más larga } = máx { 574, 213 } = 574 min; ninguna solución puede ser menor, pero tampoco se sabe si LB es alcanzable.

3. Las tres formas de codificar el cromosoma

Codificación	Cómo se representa una solución	Longitud y espacio de búsqueda
A. Binaria	Cadena de bits. Cada operación tiene un bloque de 8 bits; su valor entero (0 a 255) es la prioridad de la operación (menor valor = se atiende antes).	86 × 8 = 688 bits. Espacio de 2^688 cadenas.
B. Permutación con repetición (entera)	Vector de enteros donde cada número es una orden y aparece tantas veces como operaciones tiene (J1 aparece 6 veces, J6 aparece 5, etc.). La k-ésima aparición de la orden j representa su k-ésima operación; su prioridad es la posición en el vector.	86 genes enteros. Permutaciones de un multiconjunto; todas son válidas.
C. Llaves aleatorias (real)	Vector de números reales en [0, 1), uno por operación. La llave es la prioridad de la operación (Bean, 1994).	86 genes reales. Espacio continuo de 86 dimensiones.

Tabla 4. Codificaciones comparadas.

Se eligieron porque son las tres familias clásicas de representación (binaria, entera/permutación y real) y porque cada una exige operadores genéticos distintos. Las tres terminan produciendo siempre un calendario factible, porque la factibilidad la garantiza el decodificador y no el cromosoma.

4. Genotipos y fenotipos

El **genotipo** es el cromosoma tal como lo manipula el AG; el **fenotipo** es el calendario de producción que resulta de decodificarlo: instante de inicio y de fin de cada operación en su máquina (el diagrama de Gantt) y el makespan.

Codificación	Genotipo	Ejemplo (primeros genes)	Fenotipo
A. Binaria	Cadena de 688 bits (86 bloques de 8 bits).	01101001 11100010 00010111 … Prioridades de las operaciones 1, 2 y 3: 105, 226 y 23.	Calendario de 86 operaciones con inicio, fin y makespan.
B. Permutación con repetición	Vector de 86 números de orden (multiconjunto con n_j copias de j).	[3, 1, 3, 2, 1, 2, …] Primero la operación 1 de J3 (prioridad 1), luego la 1 de J1 (2), la 2 de J3 (3), …	Mismo tipo de calendario (diagrama de Gantt).
C. Llaves aleatorias	Vector de 86 reales en [0, 1).	[0.71, 0.08, 0.45, …] La operación con la llave más baja tiene mayor prioridad.	Mismo tipo de calendario (diagrama de Gantt).

Tabla 5. Genotipos, ejemplos y fenotipo.

Decodificación (genotipo → fenotipo). Las tres codificaciones se convierten primero en un vector de prioridades, una por operación (binaria: el entero del bloque; permutación: la posición; llaves: el valor de la llave). Después se aplica el mismo algoritmo de Giffler y Thompson (1960), que construye un calendario activo:

- Cada orden tiene una operación pendiente (la siguiente de su ruta). Para cada una se calcula el instante más temprano en que podría terminar: inicio = máx { fin de la orden, máquina libre } más su tiempo de proceso.
- Se toma la máquina m* de la operación que terminaría primero y se arma el conjunto de conflicto: operaciones pendientes en m* que podrían empezar antes de ese instante.
- De ese conjunto se programa la de mayor prioridad (menor valor en el genotipo) y se actualizan los tiempos de la orden y de la máquina. Se repite 86 veces.

Como el decodificador es el mismo, las diferencias de resultados provienen de la codificación y de sus operadores genéticos asociados, no de la forma de construir el calendario.

5. Métodos usados en cada una de las 7 etapas del algoritmo genético

Se consideraron las siguientes siete etapas del ciclo del AG: (1) codificación, (2) población inicial, (3) evaluación, (4) selección, (5) cruza, (6) mutación y (7) reemplazo y criterio de paro.

Etapa	A. Binaria	B. Permutación con repetición	C. Llaves aleatorias
1. Codificación	Cadena de 688 bits (8 bits por operación).	Vector de 86 enteros (número de orden repetido n_j veces).	Vector de 86 reales en [0, 1).
2. Población inicial	100 individuos con bits equiprobables.	100 permutaciones aleatorias del multiconjunto (Fisher–Yates).	100 vectores con U(0, 1).
3. Evaluación	Decodificar (Giffler–Thompson) y calcular C_max; aptitud 1 / C_max.	Igual que A.	Igual que A.
4. Selección	Torneo de tamaño 3 (con reemplazo).	Igual que A.	Igual que A.
5. Cruza	Dos puntos, pc = 0.9.	POX (Precedence Operation Crossover): un subconjunto aleatorio de órdenes conserva sus posiciones del padre 1 y el resto se rellena en el orden del padre 2; pc = 0.9.	Uniforme (cada gen viene de un padre al azar con p = 0.5); pc = 0.9.
6. Mutación	Inversión de bit con p = 2/688 ≈ 0.0029 por bit (unos 2 bits por individuo).	Intercambio de dos posiciones al azar con probabilidad 0.30 por individuo.	Perturbación gaussiana $N(0, 0.15^2)$ con p = 2/86 por gen, recortada a [0, 1).
7. Reemplazo y paro	Generacional con elitismo (los 2 mejores pasan intactos). Paro a las 300 generaciones (≈ 29,500 evaluaciones por corrida).	Igual que A.	Igual que A.

Tabla 6. Métodos de cada etapa por codificación. Parámetros comunes: población 100, 300 generaciones, 30 corridas independientes (semillas 1000 a 1029).

Para que la comparación sea justa, todas las codificaciones usan la misma población, selección, elitismo y número de evaluaciones. Los parámetros de mutación y cruza se fijaron con valores usuales antes de correr los experimentos y no se afinaron por separado para cada codificación.

6. Código que apoya el cumplimiento del objetivo

El programa está escrito en C# (compilado con el compilador incluido en .NET Framework 4, sin dependencias externas). Lee la instancia sintética, ejecuta el AG con las tres codificaciones y la búsqueda aleatoria, calcula las estadísticas y genera los archivos CSV y las gráficas que aparecen en este informe.

Enlace al código:
file:///C:/Users/Alonz/Documents/Programacion/LAGE/sept%20dic%202026/Actividad%2005%20-%20Algoritmo%20genetico/GaJobShop.cs

Componente (clase)	Función
Instance	Genera la instancia sintética (rutas y tiempos) con semilla fija y calcula la cota inferior.
Decoder	Convierte un vector de prioridades en un calendario (Giffler–Thompson) y devuelve el makespan.
BinaryCodec, PermCodec, KeysCodec	Una clase por codificación: individuo aleatorio, decodificación a prioridades, cruza y mutación.
Ga	Ciclo del AG (selección por torneo, elitismo, reemplazo) y la búsqueda aleatoria de referencia.
Stats y Charts	Estadísticos, prueba de Mann–Whitney y gráficas (convergencia, cajas y diagrama de Gantt).
Program	Ejecuta 30 corridas por método, las reglas de despacho y escribe los resultados en la carpeta resultados.

Tabla 7. Estructura del código (GaJobShop.cs).

Los resultados completos (CSV, gráficas y el calendario de la mejor solución) están en la carpeta resultados junto al código. Las semillas son fijas, de modo que al volver a ejecutar el programa se obtienen los mismos valores de makespan.

7. Resultados

Calidad de las soluciones. La Tabla 8 resume el makespan final de 30 corridas independientes por método (menor es mejor). La brecha se mide contra la cota inferior de 574 min.

Método	Mejor	Media	Peor	Desv. est.	Brecha media	Gen. del mejor	Tiempo (ms)
Binaria	619	639.7	671	11.48	11.5 %	162.2	450
Permutación con repetición	609	614.4	627	4.37	7.0 %	87.2	364
Llaves aleatorias (real)	610	628.3	653	9.75	9.5 %	137.1	344
Búsqueda aleatoria	672	684.6	694	6.27	19.3 %	141.1	462

Tabla 8. Makespan (min) en 30 corridas. "Gen. del mejor" es la generación promedio en que se halló la mejor solución de cada corrida (para la búsqueda aleatoria se agrupan las evaluaciones en 300 bloques). Tiempo medio por corrida de ≈ 29,500 evaluaciones.

Referencia	Makespan (min)
Cota inferior (LB)	574
Regla SPT (tiempo de proceso más corto)	921
Regla MWKR (más trabajo restante)	728
Despacho por orden de trabajos (J1, J2, …)	832
Mejor solución del AG (Permutación con repetición)	609

Tabla 9. Valores de referencia deterministas, calculados con el mismo decodificador.

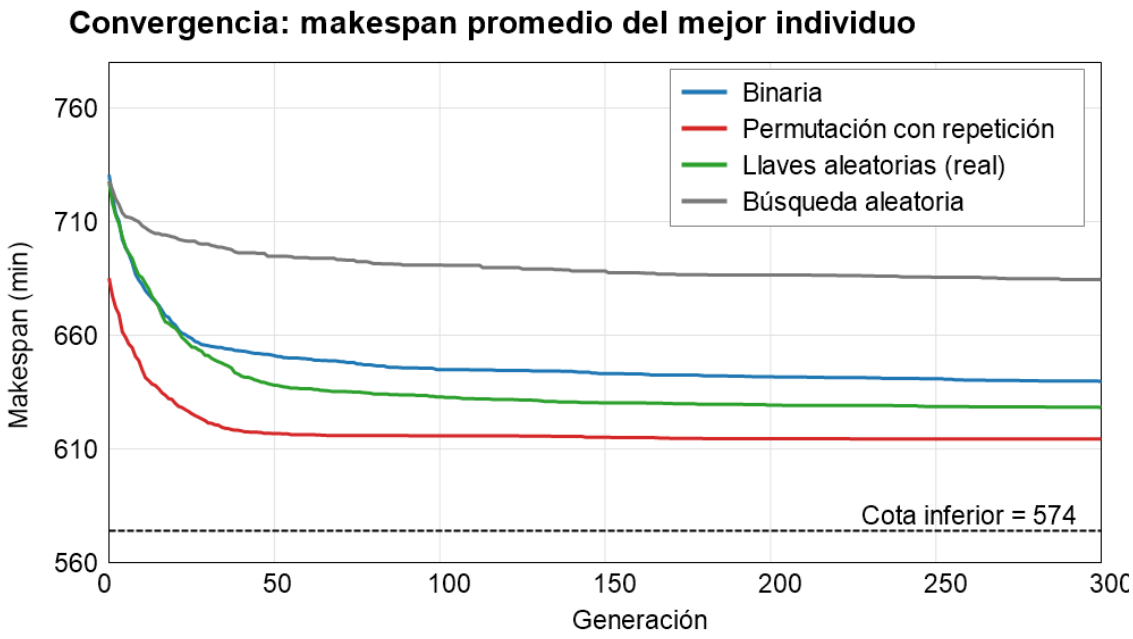


Figura 1. Convergencia: makespan promedio del mejor individuo por generación (30 corridas).

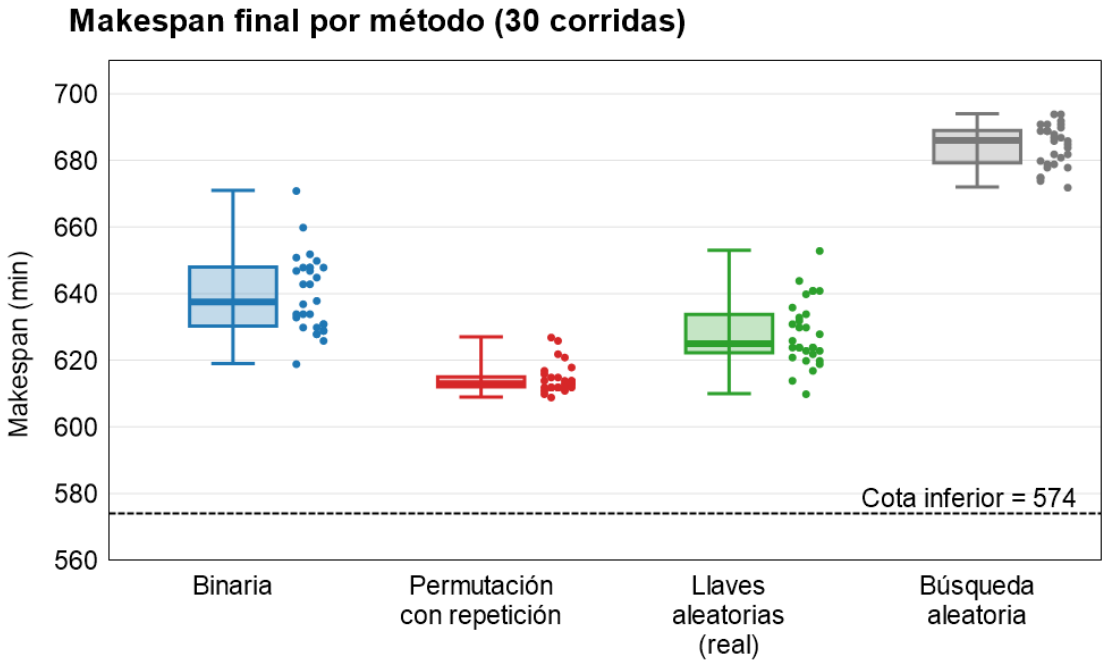


Figura 2. Distribución del makespan final por método (caja: cuartiles y mediana; puntos: cada corrida).



Figura 3. Diagrama de Gantt (fenotipo) de la mejor solución global: 609 min. Cada color es una orden de trabajo.

Evolución del mejor makespan promedio.

Método	Generación 0	Generación 50	Generación 300
Binaria	730.6	651.1	639.7
Permutación con repetición	685.2	616.9	614.4
Llaves aleatorias (real)	727.6	638.1	628.3
Búsqueda aleatoria	728.0	694.8	684.6

Tabla 10. Makespan promedio del mejor individuo en tres momentos de la evolución.

Significancia estadística. Prueba U de Mann–Whitney bilateral (aproximación normal con corrección por empates), $\alpha = 0.05$, 30 corridas contra 30:

Método A	Método B	z	Valor p	¿Diferencia significativa?
Binaria	Permutación con repetición	6.59	< 0.0001	Sí
Binaria	Llaves aleatorias (real)	3.76	0.0002	Sí
Permutación con repetición	Llaves aleatorias (real)	-5.51	< 0.0001	Sí
Binaria	Búsqueda aleatoria	-6.66	< 0.0001	Sí
Permutación con repetición	Búsqueda aleatoria	-6.67	< 0.0001	Sí
Llaves aleatorias (real)	Búsqueda aleatoria	-6.66	< 0.0001	Sí

Tabla 11. Comparaciones por pares del makespan final.

8. Interpretación y análisis de los resultados

El AG sí aporta valor. Con cualquier codificación supera por mucho a la búsqueda aleatoria con el mismo número de evaluaciones ($p < 0.0001$) y a las reglas de despacho: la media de la permutación con repetición (614.4 min) es 10.2 % menor que la de la búsqueda aleatoria, 15.6 % menor que MWKR (728 min) y 33.3 % menor que SPT (921 min). La mejor solución global (609 min) queda a 6.1 % de la cota inferior.

Ranking de codificaciones. Quedó así: permutación con repetición (media 614.4, desv. 4.37), llaves aleatorias (628.3, desv. 9.75) y binaria (639.7, desv. 11.48). Las tres diferencias son estadísticamente

significativas (Tabla 11); la menor es binaria contra llaves ($p = 0.0002$). La permutación además es la más estable: su peor corrida (627 min) es mejor que la media de las otras dos codificaciones.

Por qué ganó la permutación (explicaciones probables, no probadas con experimentos adicionales).

- Su genotipo ya es un orden de procesamiento: los cambios que hacen la cruza POX y el intercambio se traducen en cambios de secuencia pequeños y con sentido, y POX conserva el orden relativo de las órdenes que hereda de cada padre (bloques constructivos).
- En la binaria, un solo bit de alto peso cambia drásticamente la prioridad de una operación, y la cruza de dos puntos corta bloques sin relación con la estructura de las órdenes. Además, 688 bits son una representación muy redundante: sólo importa el orden relativo de las prioridades, no su valor exacto.

En las llaves aleatorias la mutación gaussiana permite ajustes finos y la cruza uniforme no rompe bloques, por eso queda a medio camino; sin embargo también es redundante (muchos vectores distintos producen la misma secuencia) y su espacio continuo es más lento de explorar. La población inicial de la permutación ya parte de un mejor makespan (685.2 contra ≈ 730.6 de las otras): al ordenar por posición, las operaciones de una misma orden reciben prioridades crecientes, lo que favorece que cada orden avance de forma continua por su ruta (hipótesis).

Convergencia y costo. La permutación converge más rápido (generación media del mejor: 87.2 contra 137.1 de las llaves y 162.2 de la binaria) y casi se estabiliza antes de la generación 60 (Figura 1), lo que sugiere riesgo de convergencia prematura; la binaria seguía mejorando lentamente al terminar. El costo de cómputo es similar (344 a 450 ms por corrida) porque casi todo el tiempo lo consume el decodificador, que es común; la binaria cuesta un poco más por el paso de bits a enteros.

Limitaciones. (1) Se usó una sola instancia sintética; el ranking podría cambiar con otros tamaños o cargas. (2) No se afinaron los parámetros para cada codificación (la binaria y las llaves podrían mejorar con otras tasas de mutación). (3) La comparación mide codificación y operadores juntos, tal como se usan en la práctica, no la codificación aislada. (4) Ninguna corrida alcanzó la cota inferior; no se sabe si el óptimo está por debajo de 609 min. (5) Los tiempos dependen del equipo.

Recomendaciones. Para programar un taller real conviene usar permutación con repetición y POX; como mejoras se sugieren aumentar la diversidad (mayor mutación o reinicios), hibridar con búsqueda local y probar con más instancias y con tiempos de preparación o fechas de entrega.

Conclusión

Se aplicó un algoritmo genético a un taller de maquinado de 15 órdenes, 6 máquinas y 86 operaciones, comparando las codificaciones binaria, de permutación con repetición y de llaves aleatorias bajo las mismas condiciones. Las tres mejoran claramente a la búsqueda aleatoria y a las reglas de despacho, pero la permutación con repetición obtuvo el mejor y más estable resultado (media 614.4 min, mejor 609 min, 6.1 % sobre la cota inferior), seguida de las llaves aleatorias y de la binaria; las diferencias son significativas. La forma de codificar el cromosoma, por tanto, influye tanto como el resto de los operadores: conviene que el genotipo exprese directamente la estructura del problema (aquí, un orden de procesamiento).

Referencias bibliográficas

Bean, J. C. (1994). Genetic algorithms and random keys for sequencing and optimization. ORSA Journal on Computing, 6(2), 154–160.

Bierwirth, C. (1995). A generalized permutation approach to job shop scheduling with genetic algorithms. OR Spektrum, 17(2–3), 87–92.

Cheng, R., Gen, M., y Tsujimura, Y. (1996). A tutorial survey of job-shop scheduling problems using genetic algorithms—I. Representation. Computers & Industrial Engineering, 30(4), 983–997.

Davis, L. (1985). Job shop scheduling with genetic algorithms. En Proceedings of the First International Conference on Genetic Algorithms and their Applications (pp. 136–140). Lawrence Erlbaum.

Eiben, A. E., y Smith, J. E. (2015). Introduction to Evolutionary Computing (2.ª ed.). Springer.

Garey, M. R., Johnson, D. S., y Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. Mathematics of Operations Research, 1(2), 117–129.

Giffler, B., y Thompson, G. L. (1960). Algorithms for solving production-scheduling problems. Operations Research, 8(4), 487–503.

Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley.

Gonçalves, J. F., y Resende, M. G. C. (2011). Biased random-key genetic algorithms for combinatorial optimization. Journal of Heuristics, 17(5), 487–525.

Holland, J. H. (1975). Adaptation in Natural and Artificial Systems. University of Michigan Press.

Mann, H. B., y Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. The Annals of Mathematical Statistics, 18(1), 50–60.

Nakano, R., y Yamada, T. (1991). Conventional genetic algorithm for job shop problems. En Proceedings of the Fourth International Conference on Genetic Algorithms (pp. 474–479). Morgan Kaufmann.

Pinedo, M. L. (2016). Scheduling: Theory, Algorithms, and Systems (5.ª ed.). Springer.